

CANDY

DIALOGUE ENGINE

Condition Commands

1.0.0

Condition commands are used when you need dialogue to run specific lines or branch in different directions depending on conditions.

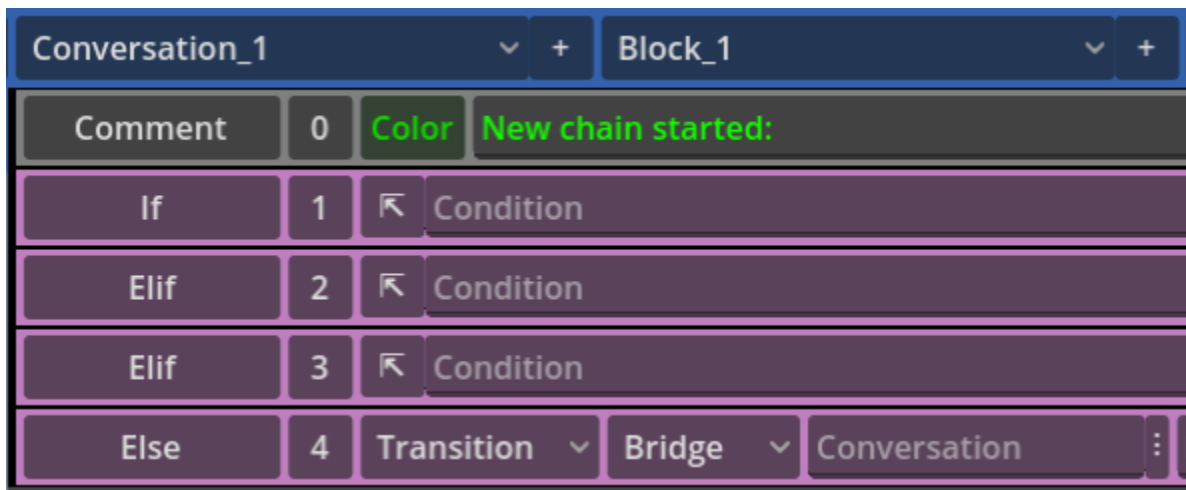
Conditions

`§If` / `§Elif` / `§Else` Chains

The `§If`, `§Elif` and `§Else` commands can be used very similarly to `if` / `elif` / `else` in GDScript.

Add an `§If` command to start a chain, indicate the condition that must be fulfilled and what should happen when it is, and that's it.

Then, add `§Elif` commands or an `§Else` command in immediate succession, as needed.



Keep in mind that `§Elif` and `§Else` will only work if the chain is not broken prior: `§If` starts a chain, and the chain will break when it encounters any spoken line or any command other than `§Elif`, `§LM` or `§Comment`.

Conversation_1		▼	+	Block_1		▼	+
Comment	0	Color	New chain started:				
If	1	↶	Condition				
Comment	2	Color	Chain Broken:				
Call	3	Await	Function	Arguments			
Elif	4	↶	Condition				
Else	5	Transition	▼	Bridge	▼	Conversation	⋮

Just like in GDScript, \$Else will break a chain, while adding another \$If command will start a new one.

\$LM and \$Comment are exempted from breaking chains because they are essentially just information inserted in the dialogue script, not actual commands that have any effect on their own.

Note that placing an \$LM command in the middle of a condition chain serves no purpose: if you transition to that \$LM directly, the \$If that started the chain won't be processed. Therefore, any \$Elif or \$Else after the \$LM will fail because no chain will have been started. See the Transitions guide for details about the \$LM command and transitions.

\$For & \$While Loops

Candy DE also provides \$For and \$While commands to emulate GDScript for and while loops.

For	5	↶	Condition	Transition
While	6	↶	Condition	Transition

They work similarly to \$If, \$Elif and \$Else: write the condition that must be fulfilled, and indicate what happens when it is.

The \$While command has a limit to the number of loops it can perform! This is an optional safety feature to avoid accidental infinite looping. By default, it is set at 1000 loops.

To change the loop limit, or disable it altogether, open [Candy_Properties.gd](#) and look for **while_safety_break**. Set it to a negative value to disable it.

```
var while_safety_break = 1000
```

Condition Writing

Candy DE uses Godot's native `Expression.execute()` method to parse comparison strings.

To write conditions, just write them like you would in GDScript, with one small distinction: don't write 'if', 'elif', 'else', 'for' or 'while' -- the engine knows what type of condition you're using by looking at the command.

You'll also need to use the variable symbols (default '£', '\$' and '€') to designate variables (see the [Variables](#) guide for details).

As an example:

£globals.player_health < 10

You can compare variables to one another, use 'AND/and/&&', 'OR/or/||' and 'not/!' operators, and even use parentheses:

£globals.var_1 == £globals.var_2 OR (£globals.var_1 > 3 AND globals.var_2 == true)

'NOT' (uppercase) is not supported, and neither is 'XOR/xor/^'

Note that the Expression class may have limitations over regular GDScript. Also, at this time, the official Godot documentation doesn't seem to mention condition operators (we may have missed something); however, the Expression class is confirmed working for comparisons by our own testing.

Note also that you can't chain comparisons inside an expression. For example, this isn't possible:

2 < £globals.var_1 < 8

This will fail because `Expression.execute()` will first convert '2 < £globals.var_1' to 'true'; this then results in comparing 'true < 8' which isn't possible.

Instead, write:

£globals.var_1 > 2 AND £globals.var_1 < 8

Condition Editor

Typing long conditions in a single line isn't ideal. If you click the "⌵" button next to the Condition field in any condition command, a window opens with a bigger text area.

Syntax: £autoload.variable \$node_path.variable €dialogue_variable

CLOSE

Condition

You can safely add line breaks for convenience: Candy DE will temporarily replace them with spaces when it processes the condition, allowing it to read the string perfectly.

Super Variables

Instead of providing a condition string directly to a condition command, you can store an entire condition string in a variable, then reference that variable in the command data. For example:

```
var condition_1 = "£globals.player_hp >= 100"
```

However, since the engine expects to find variables inside the condition string by default, the base variable symbols (default: £, \$ and €) will not work for replacing the entire string with a variable reference.

For example, if you just write "£globals.condition_1" in the condition string, the engine will read this as an incomplete condition: it expects a value to compare the variable to, and an operator to tell it what kind of comparison to make.

Instead, you must use the 'super' variable symbols (default: '££', '\$\$' and '€€'), like this:

```
££globals.condition_1
```

This will make the engine replace the entire condition string with the value of condition_1 above:

```
£globals.player_hp >= 100
```

Consequences

To define what happens when a condition is fulfilled, use the drop-down menu to the right of the "Condition" field, which says "Transition" by default.

This menu lets you choose between several options:

Set → Modify the value of a variable, similar to the §Set command.

Call → Call a function, similar to the §Call command.

Emit → Emit a signal, similar to the §Emit command.

Await → Await a signal, similar to the `§Await` command.

Flag → Change a flag's value, similar to the `§Flag` command.

Name → Change an actor's display name, similar to the `§Name` command.

Disposition → Change an actor's disposition, similar to the `§Disposition` command.

Role → Change a role's assigned actor, similar to the `§Role` command.

Transition → Transition to another Block, similar to `§Jump`, `§Bridge`, `§Return` or `§End`.

For details about these commands, see the [Control Commands](#) guide and the [Transitions](#) guide.

The Transition option is the one you'll use when you want a condition to run any other command, or multiple commands, or even display Spoken Lines.

For example, if you want a character to say something when a condition is fulfilled, set transition to a block that features the spoken lines you wish.

If you use a 'Bridge' transition, the condition will trigger a brief detour: the Block it transitions to will be processed, then the engine will return to the initial Block and continue after the condition command.

For permanent transitions, use 'Jump' instead.

The 'Return' option returns immediately to a previous Block, but only if the current Block was accessed with a 'Bridge' transition: if not, 'Return' ends the dialogue entirely -- just like the 'End' option also does.

You can of course nest condition commands: simply add more condition commands in the Block that a condition command transitions to.

Potential Errors

Setting up conditions isn't difficult, in fact it's very intuitive. But just like when coding, you'll need to be careful what you actually make your conditions do. The most likely errors would be incorrectly chaining `§If`, `§Elif` and `§Else` commands, or using the wrong transition type.

If you regularly test your dialogues while you work, you'll immediately notice such errors and you'll be able to easily correct them.